ensia The National School of Artificial Intelligence المدرسة الوطنية العليا للذكاء الاصطناعي

Introduction to Blockchain

Lab #3

Implementing a Simplified Cryptocurrency Transaction System

Objective:

- Understand how cryptocurrency transactions work internally.
- Implement UTXO (Unspent Transaction Output) logic.
- Create and verify signed transactions using ECDSA.
- Prevent double spending.
- Simulate a few users sending coins to each other.

Instructions:

In Bitcoin, the blockchain records *blocks of transactions*.
Each transaction consumes previous unspent outputs (UTXOs) and creates new ones.
Ownership of coins is proven using **digital signatures**.

In this lab, you'll simulate **the transaction layer only**, assuming the blockchain infrastructure already exists.

Lab Setup

Language: Python 3
Libraries:

```
pip install ecdsa
```

Part 1 — Wallets and Addresses

Each user (Alice, Bob, etc.) has:

- A **private key** → used to sign transactions.
- A **public key** → used to verify signatures.
- An **address** → derived from the public key.

Task 1:

Generate ECDSA key pairs and print each user's address.

```
from ecdsa import SigningKey, SECP256k1
import hashlib

def sha256_hex(b): return hashlib.sha256(b).hexdigest()

def generate_wallet():
    sk = SigningKey.generate(curve=SECP256k1)
    vk = sk.get_verifying_key()
    address = sha256_hex(vk.to_string())[:40]
    return sk, vk, address
```

Part 2 — Transactions and UTXO Model

Each transaction has:

- **Inputs:** references to previous unspent outputs (tx_id, index, signature).
- **Outputs:** new coins (recipient address + amount).

A **UTXO pool** keeps track of which outputs are spendable.

Task 2:

Create a transaction structure and a UTXO pool.

```
class TxOutput:
    def __init__(self, value, recipient):
        self.value = value
        self.recipient = recipient

class TxInput:
    def __init__(self, prev_tx, prev_index, signature, pubkey):
        self.prev_tx = prev_tx
        self.prev_index = prev_index
        self.signature = signature
        self.pubkey = pubkey

class Transaction:
    def __init__(self, inputs, outputs):
        self.inputs = inputs
        self.outputs = outputs

class UTXOPool:
    def __init__(self):
        self.utxos = {}  # (tx_id, index): TxOutput
```

Part 3 — Coinbase and Initial Coins

The system starts with one **coinbase transaction** that creates some coins for the miner (or “bank”).

Task 3:

Create a coinbase transaction that adds coins to Alice’s wallet.

```
import json, hashlib

def tx_hash(tx):
    return hashlib.sha256(json.dumps(tx,
sort_keys=True).encode()).hexdigest()

# Coinbase: creates new coins without inputs
coinbase_tx = {
```

```
    "inputs": [],
    "outputs": [{"value": 50, "recipient": "Alice"}]
}
txid = tx_hash(coinbase_tx)
utxos = {(txid, 0): TxOutput(50, "Alice")}
```

Part 4 — Signing and Verifying Transactions

Task 4:

When Alice sends coins to Bob, she must sign the transaction with her private key.

```
def sign(sk, message):
    return sk.sign(message.encode()).hex()

def verify(vk, signature_hex, message):
    try:
        vk.verify(bytes.fromhex(signature_hex), message.encode())
        return True
    except:
        return False
```

Alice signs her transaction:

```
message = "Alice->Bob:30"
signature = sign(alice_sk, message)
```

Part 5 — Validating Transactions

A valid transaction must:

1. Use only unspent outputs (UTXOs).
2. Be signed by the rightful owner.

3. Not spend more than available.

4. Remove used UTXOs and add new ones.

```
def validate_transaction(tx, utxos):
    total_in, total_out = 0, 0

    for i, inp in enumerate(tx.inputs):
        key = (inp.prev_tx, inp.prev_index)
        utxo = utxos.get(key)
        if not utxo:
            print("Invalid input: spent or unknown UTXO")
            return False
        addr = sha256_hex(bytes.fromhex(inp.pubkey))[:40]
        if addr != utxo.recipient:
            print("Signature does not match owner.")
            return False
        if not
verify(VerifyingKey.from_string(bytes.fromhex(inp.pubkey),
curve=SECP256k1),
                       inp.signature, tx_hash(tx.to_dict())):
            print("Invalid signature")
            return False
        total_in += utxo.value

    total_out = sum(o.value for o in tx.outputs)
    if total_out > total_in:
        print("Overspending!")
        return False
    return True
```

Part 6 — Simulation Scenario

Steps:

1. Coinbase gives **Alice 50 coins**.

2. Alice sends **30 to Bob** and keeps **20 as change**.
3. Bob sends **10 to Carol**.
4. Try to make Alice double-spend the same UTXO — the program should reject it.

Expected output:

```
Transaction 1: Alice → Bob (30), Change: 20 ✅
Transaction 2: Bob → Carol (10), Change: 20 ✅
Transaction 3: Alice double-spend ❌ Rejected
```